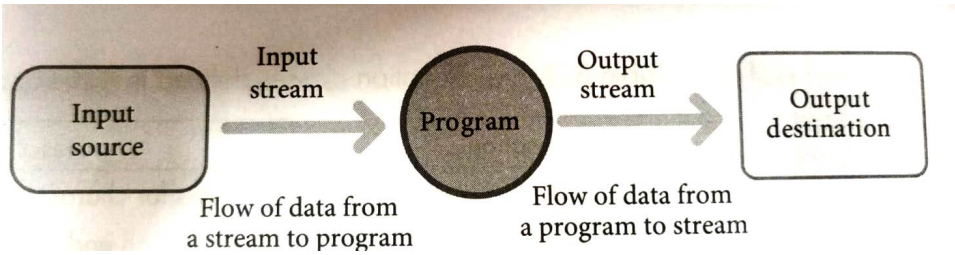
Streams:

Java treats flow of data as stream. Input and output in JAVA is based on the concept of stream. A stream is an abstract representation of an input device(produces data) or output device(consumes data).

The flow of data from a source to a Java program is called as input stream and the flow of data from a Java program to destination is called output stream. The java.io package contains a large number of stream classes to support the streams.



Standard IO Streams:

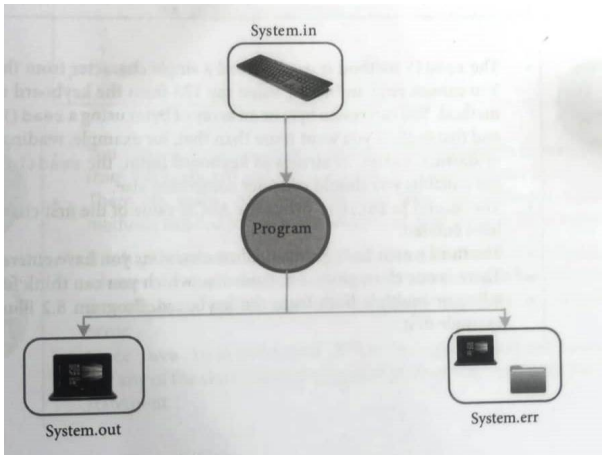
Java defines three standard streams that are accessible though members of System class.

1. **Standard input stream**: This is referred by System.in and usually corresponds to the keyboard by default. This is encapsulated by in member of System class and is of type InputStream.

2. **Standard output stream**: This is referred by System.out and usually corresponds to the console(display screen of the computer). This is encapsulated by out member of System class and is of type PrintStream.

3. **Standard error stream**: This is referred by System.err, is used to display error messages and usually corresponds to the console(display screen of the computer). This is encapsulated by err member of System class and is of type PrintStream.

SURESH YADLAPATI



Ex: Reading some data using System.in stream:

```
import java.io.*;
class StandardInputDemo {
   public static void main(String args[]) throws IOException {
      System.out.println("Enter whatever you want...");
      int i = System.in.read();
      System.out.println(i);
      System.out.println((char) i);
   }
}
```

Ex2: Reading lines of data from key board

```
import java.io.Console;

public class ConsoleDemo {
   public static void main(String[] args) {
      Console console = null;
      String name = null;

      try {
         consol = System.console();

         /*The System.console() method returns an instance  of the Console class,
           which provides methods to read user input and print text directly to the
            console.       */

           if (consol != null) {
                System.out.println(“Your name please“);
                 name = consol.readLine("Name: ");
                  System.out.println("Hi  " + name + “!”);

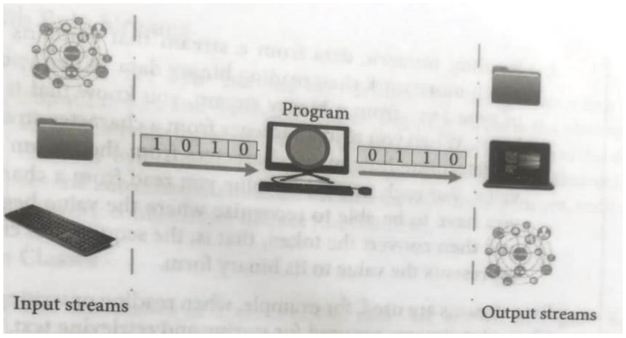
         }
      } catch (Exception ex) {
         ex.printStackTrace();
      }
   }
}
```

SURESH YADLAPATI

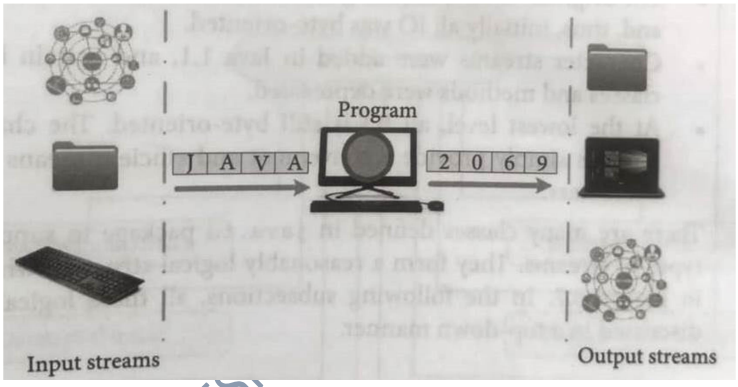
Types of Streams:

A stream can be used broadly in two ways: Byte streams and Character streams.

- Byte streams considers data in binary representation.



- Character streams read data in the form of characters as a unit. Java stores internally characters as Unicode characters which are 16 bit in size.

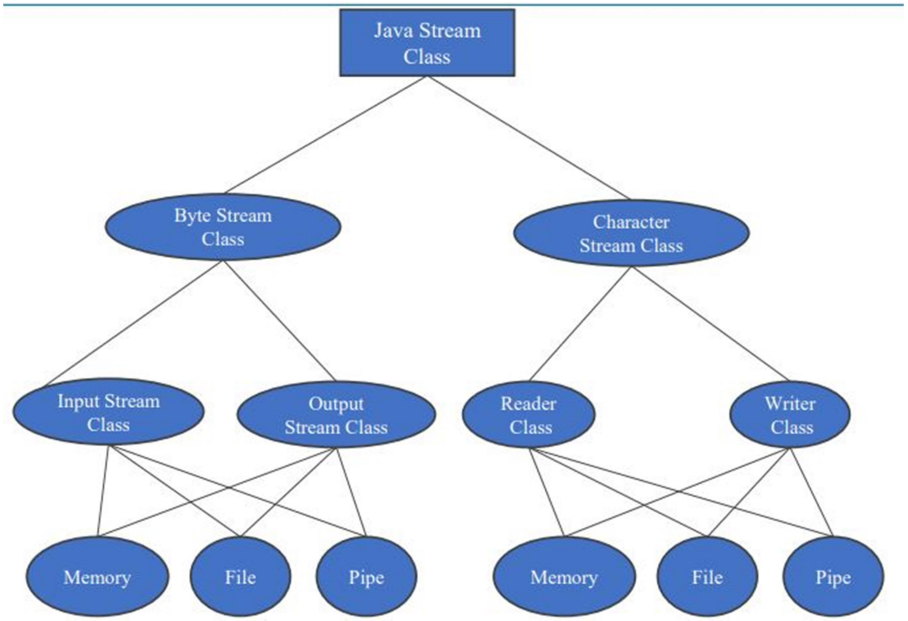


Differences between byte and character stream classes:

Aspect	**Character Streams**	**Byte Streams**
Data Handling	Handle character-based data	Handle raw binary data
Representation	Classes end with "Reader" or "Writer"	Classes end with "InputStream" or "OutputStream"
Suitable for	Textual data, strings, human-readable info	Non-textual data, binary files, multimedia
Character Encoding	Automatic encoding and decoding	No encoding or decoding
Text vs non-Text data	Text-based data, strings	Binary data, images, audio, video
Performance	Additional conversion may impact performance	Efficient for handling large binary data
Handle Large Text Files	May impact performance due to encoding	Efficient, no encoding overhead
File Handling	Read/write text files	Read/write binary files

Network Communication	Sending/receiving text data	Sending/receiving binary data
Handling Images/Audio/Video	Not designed for handling binary data directly	Suitable for handling binary multimedia data

The logical structure of java.io package is shown in the following figure:

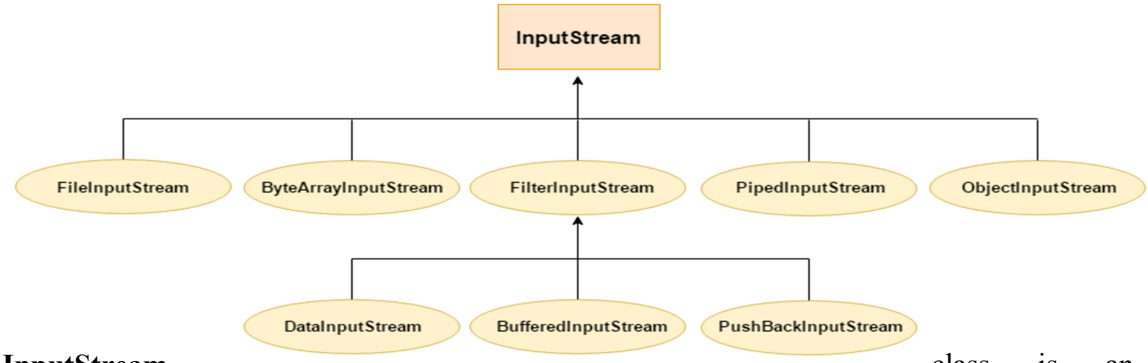


Byte Streams:

In java.io, byte streams are defined by using two class hierarchies: InputStream and OutputStream.

InputStream is the abstract base class for byte stream input operations whereas OutputStream is the base class for byte stream operations.

InputStream Classes:



InputStream class is an abstract base class for byte stream input operations. InputStream class contains the following methods which are implemented in sub classes.

- int read() - reads one byte of data from the input stream
- int read(byte[] array) - reads bytes from the stream and stores in the specified array
- int read(byte[] array, int offset, int numBytes) – Attempt to read numBytes into array starting at offset.
- int available() - returns the number of bytes available in the input stream
- void mark(int numberofbytes) - marks the position in the input stream up to which data has been read
- void reset() - returns the control to the point in the stream where the mark was set
- Boolean markSupported() - checks if the mark() and reset() method is supported in the stream
- long skips(long numberofbytes) - skips and discards the specified number of bytes from the input stream
- void close() - closes the input stream

ByteArrayInputStream: Input stream that reads from a byte array. An instance of this contains internal buffer.

FileInputStream: Input stream that reads from a file.

ObjectInputStream: For reading objects from a stream.

PipedInputStream: For reading from a piped stream like a network socket or port.

FilterInputStream: For filtering input from an existing stream.

DataInputStream: An input stream that contains methods for reading Java primitive data types from stream.

Methods in DataInputStream class:

- **boolean readBoolean()**: Reads one byte and returns true if it's nonzero, false otherwise.
- **byte readByte()**: Reads and returns one byte (8-bit signed integer).
- **char readChar()**: Reads two bytes and returns a char value (16-bit Unicode character).
- **double readDouble()**: Reads eight bytes and returns a double value.
- **float readFloat()**: Reads four bytes and returns a float value.
- **int readInt()**: Reads four bytes and returns an int (32-bit signed integer).
- **long readLong()**: Reads eight bytes and returns a long (64-bit signed integer).
- **short readShort()**: Reads two bytes and returns a short (16-bit signed integer).
- **String readUTF()**: Reads a UTF-8 encoded string.

BufferedInputStream: Reading a chunk of bytes from an internal buffer associated with a stream.

SURESH YADLAPATI

PushbackInputStream: It can unread a byte which is already read and push back one byte.

Example program 1:

```
import java.io.*;
public class InputStreamMethods {
        public static void main(String[] args) throws IOException {
      FileInputStream in = new FileInputStream("input.txt");
      // 1. read()
      System.out.println("1. read(): " + (char) in.read());
      // 2. read(byte[] b)
      byte[] b = new byte[5];
      System.out.println("2. read(byte[]):" +  in.read(b)+ ":"+new String(b));
      byte[] c = new byte[5];
      System.out.println("2. read(byte[], 0, n):" + in.read(c,0,2)+":"+new String(c));
      // 3. available()
      System.out.println("3. available(): " + in.available());
       // 4. skip(n)
      in.skip(1);
      System.out.println("4. skip(1): One byte skipped");
      System.out.println("1. read(): " + (char) in.read());
     // 5. read(byte[] b, int off, int len)
      byte[] d = new byte[10];
      System.out.println("5. read(d, 0, 2): "+ in.read(d, 0, 2)+":"+new String(d));
      // 6. markSupported()
      System.out.println("6. markSupported(): " + in.markSupported());
      // 7. close()
      in.close();
      System.out.println("7. close(): Stream closed");
   }
}
```

SURESH YADLAPATI

Example program 2:

```
import java.io.*;
public class ExampleonMark {
   public static void main(String[] args) throws Exception {
        FileInputStream file = new FileInputStream("input.txt");
      BufferedInputStream in = new BufferedInputStream(file);

      // Check mark support
      System.out.println("Mark Supported: " + in.markSupported());
      // Step 1: Read first 5 characters
```

```
        System.out.print("First Read: ");
        for (int i = 0; i < 5; i++) {
            System.out.print((char) in.read());
        }

        System.out.println();
        // Step 2: Mark current position
        in.mark(10);
        System.out.println("Mark placed!");

        // Step 3: Read next 4 characters
        System.out.print("Second Read: ");
        for (int i = 0; i < 4; i++) {
            System.out.print((char) in.read());
        }

        System.out.println();
        // Step 4: Return to marked position
        in.reset();
        System.out.println("Reset performed!");

        // Step 5: Read the same 4 characters again
        System.out.print("Third Read: ");
        for (int i = 0; i < 4; i++) {
            System.out.print((char) in.read());
        }

        System.out.println();
        in.close();
    }
}
```

input.txt

Suresh Yadlapati PVPSIT

```
F:\Courses\JAVA26-27\U
Mark Supported: true
First Read: Sures
Mark placed!
Second Read: h Ya
Reset performed!
Third Read: h Ya
```

SURESH YADLAPATI

Example Program 3:

```
import java.io.*;
class DIS_Example
{
```

```
public static void main(String args[])  throws IOException
{
 DataInputStream dis = new DataInputStream(System.in);
 System.out.println("Enter your name: ");
 String str1 = dis.readLine();
 System.out.println("I know your name is " + str1);

 System.out.println("Enter a whole number: ");

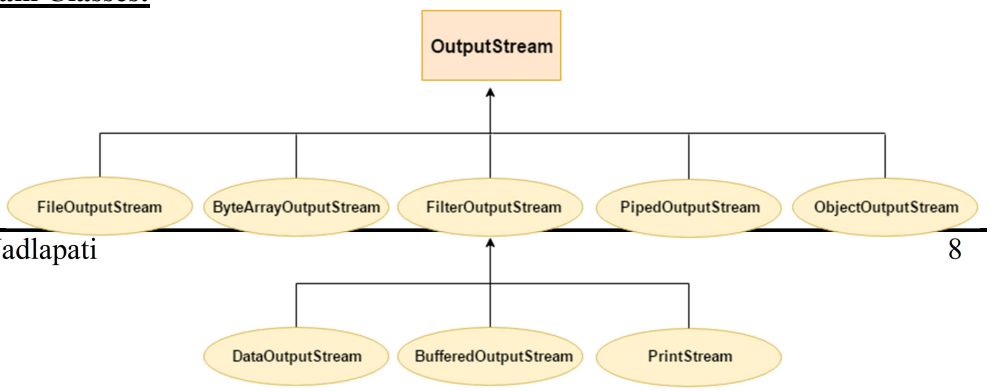
 int x = dis.readInt() ;
 System.out.println((int)x);
       /*The problem is that readInt does not behave as you might expect.
        it reads the input as  bytes i.e, 4 bytes so to avoid this use
        readLine and parse the data
       */
 System.out.println("Enter a whole number: ");

 String str2 = dis.readLine();
 System.out.println(Integer.parseInt(str2));
 dis.close();
 }
}
```

```
Enter your name:
Suresh
I know your name is Suresh
Enter a whole number:
22
842140938
Enter a whole number:
33
33
```

SURESH YADLAPATI

OutputStream Classes:



OutputStream class is an abstract base class for byte stream output operations. OutputStream class contains the following methods which are implemented in sub classes.

The methods avaibale in OutputStream class are:

Method	Description
void write(int) throws IOException	It is used to write a byte to the current output stream.
void write(byte[]) throws IOException	It is used to write an array of byte to the current output stream.
void write(byte[] arr, int offset, int numberofbytes) throws IOException	Write the numberofbytes from arr starting from the location offset.
void flush() throws IOException	It flushes the current output stream.
void close() throws IOException	It is used to close the current output stream.

SURESH YADLAPATI

FileOutputStream: Java FileOutputStream is an output stream used for writing data to a file.

ByteArrayOutputStream: Output stream that writes to a byte array. ByteArrayOutputStream class is used to write common data into multiple files. In this stream, the data is written into a byte array which can be written to multiple streams later.

PipedOutputStream: For writing to a piped stream like network.

ObjectOutputStream: For writing objects onto a stream.

FilterOutputStream: For filtering output to an existing stream.

BufferedOutputStream: Java BufferedOutputStream class is used for buffering an output stream. It internally uses buffer to store data. It adds more efficiency than to write data directly into a stream.

PrintStream: The PrintStream class provides methods to write various data values convinietly using print() and println().

DataOutputStream: An output stream that contains methods for wrinting the standard Java data types.

Methods:

writeShort()	writeDouble()	writeInt()
writeLine()	writeLong()	writeChar()
writeFloat()	writeBoolean()	writeUTF()

Example program using DataOutputStream class:

```
import java.io.*;

class DOS_Example
{
	public static void main(String[] ar) throws IOException
	{
		DataInputStream dis1 = new DataInputStream(System.in);
		System.out.println("Enter your name: ");
		String name = dis1.readLine();
		System.out.println("I know your name is " + name);

		DataOutputStream dos =new DataOutputStream(new
				FileOutputStream("pvp.txt"));

		dos.writeInt(123);
		dos.writeDouble(123.45);
		dos.writeLong(78222229);
		dos.close();

		DataInputStream dis = new DataInputStream(
		new FileInputStream("pvp.txt"));

		int   int123     = dis.readInt();
		double float12345 = dis.readDouble();
		long  long789    = dis.readLong();

		dis.close();

		System.out.println("int    = " + int123);
		System.out.println("float  = " + float12345);
		System.out.println("long    = " + long789);

	}
```

SURESH YADLAPATI

```
}
```

Example program on BufferedInputStream class:

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class MarkResetExample {
   public static void main(String[] args) {
      try (FileInputStream fis = new FileInputStream("example.txt");
           BufferedInputStream bis = new BufferedInputStream(fis)) {

         // Check if mark and reset are supported
         if (bis.markSupported()) {
            System.out.println("Mark and reset are supported.");

            // Read the first byte
            System.out.print((char) bis.read());

            // Mark the current position with a read limit of 10 bytes
            bis.mark(10);

            // Read the next few bytes
           System.out.print((char) bis.read());
          System.out.print((char) bis.read());


           // Reset back to the marked position
              bis.reset();
            // bis.skip(1);
             // Read again from the marked position
             System.out.print((char) bis.read());
             System.out.print((char) bis.read());
         } else {
            System.out.println("Mark and reset not supported in this stream.");
         }

      } catch (IOException e) {
         e.printStackTrace();
      }
   }
```

SURESH YADLAPATI

}

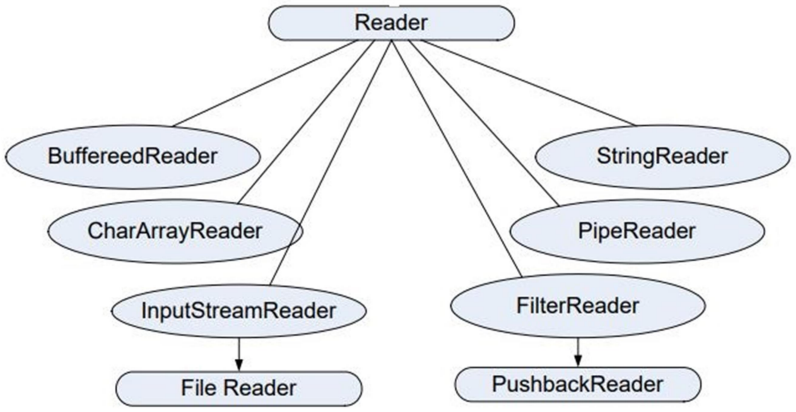
IO with Characte streams:

Character stream classes are used to read and write characters and supports a number of input-output related methods. There are two types of classes:

i. Reader stream classes

ii. Writer stream classes

Reader stream classes:



Reader: Java Reader is an abstract class for reading character streams. The methods available in Reader class are:

SURESH Y.

- int read() - reads one character of data from the input stream
- int read(char[] array) - reads characters from the stream and stores in the specified array
- int read(char[] array, int offset, int numChars) – Attempt to read numChars into array starting at offset.
- void mark(int numberofchars) - marks the position in the input stream up to which data has been read
- void reset() - returns the control to the point in the stream where the mark was set
- Boolean markSupported() - checks if the mark() and reset() method is supported in the stream
- long skips(long numberofchars) - skips and discards the specified number of characters from the input stream
- void close() - closes the input stream

BufferedReader: Java BufferedReader class is used to read the text from a character-based input stream. It can be used to read data line by line by readLine() method.

Method	Description
int read()	It is used for reading a single character.
int read(char[] cbuf, int offset, int len)	It is used for reading characters into an array
String readLine()	It is used for reading a line of text.
boolean ready()	It is used to test whether the input stream is ready to be read.
long skip(long n)	It is used for skipping the characters.
void close()	Closes the stream

CharArrayReader: The CharArrayReader is composed of two words: CharArray and Reader. The CharArrayReader class is used to read character array as a reader (stream).

InputStreamReader: InputStreamReader is a bridge from byte streams to character streams: It reads bytes and decodes them into characters using a specified charset.

FileReader: Java FileReader class is used to read data from the file.

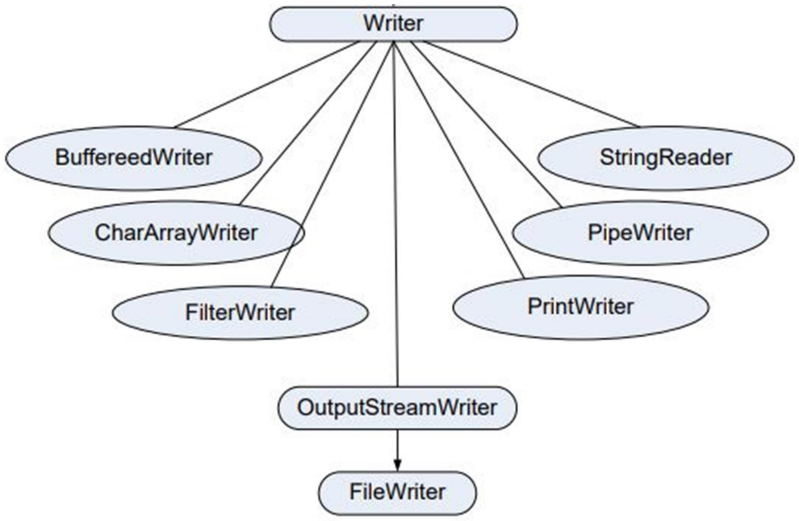
StringReader: Java StringReader class is a character stream with string as a source. It takes an input string and changes it into character stream.

PipedReader: The PipedReader class is used to read the contents of a pipe as a stream of characters.

FilterReader: Java FilterReader is used to perform filtering operation on reader stream. It is an abstract class for reading filtered character streams.

PushbackReader: Java PushbackReader class is a character stream reader. It is used to pushes back a character into stream and overrides the FilterReader class.

Writer stream classes:



Writer: It is an abstract class for writing to character streams.

Methods of Writer class:

- **void write(int c)** - Writes a single character (represented by an integer).
- **void write(char[] cbuf)** - Writes an array of characters to the output stream.
- **void write(char[] cbuf, int off, int len)** - Writes a portion of a character array, starting at off and writing len characters.
- **void write(String str**) - Writes a String to the output stream.
- **Writer append(CharSequence csq)** - Appends a character sequence (like a String or StringBuilder) to the writer. This is a convenient method for adding characters to the stream.
- **void flush()** - Flushes the stream, ensuring that any buffered characters are written out. This is especially useful with buffered streams like BufferedWriter
- **void close()** - Closes the stream and releases any system resources associated with it. After calling this method, no more data can be written to the stream.

BufferedWriter: Java BufferedWriter class is used to provide buffering for Writer instances

CharArrayWriter: The CharArrayWriter class can be used to write common data to multiple files.

FilterWriter: Java FilterWriter class is an abstract class which is used to write filtered character streams.

StringReader: Java StringReader class is a character stream with string as a source. It takes an input string and changes it into character stream.

PipedWriter: The PipedWriter class is used to write java pipe as a stream of characters.

PrintWriter : PrintWriter is a class used to write any form of data e.g. int, float, double, String or Object in the form of text either on the console or in a file in Java.

OutputStreamWriter: OutputStreamWriter is a class which is used to convert character stream to byte stream, the characters are encoded into byte using a specified charset. write() method calls the encoding converter which converts the character into bytes. The resulting bytes are then accumulated in a buffer before being written into the underlying output

stream. We optimize the performance of OutputStreamWriter by using it with in a BufferedWriter so that to avoid frequent converter invocation

FileWriter: FileWriter class is used to write character-oriented data to a file.

Example program using InputStreamReader:

```
import java.io.*;
class ISR_Example
{
        public static void main(String args[]) throws IOException
        {
                InputStreamReader isr=new InputStreamReader(System.in);
                System.out.println("Enter your name: ");
                int n  =isr.read();
                System.out.println("Character: "+ (char) n);
   }
}
```

Example program using BufferedReader class:

```
import java.io.*;
class BR_Example
{
        public static void main(String args[]) throws IOException
        {
                InputStreamReader r=new InputStreamReader(System.in);
                BufferedReader br =new BufferedReader(r);

                System.out.println("Enter your name: ");
                String name = br.readLine();

                System.out.println("Enter your age: ");
                int age = Integer.parseInt(br.readLine());

                System.out.println("Enter your Id: ");
                br.skip(1);
                int id = Integer.parseInt(br.readLine());

                System.out.println("Name: "+ name);
                System.out.println("Age: "+ age);
                System.out.println("Id: "+ id);
    }
}
```

File class:

The File class is an abstract representation of a file or directory path in the filesystem. This class allows you to manipulate files and directories (e.g., create, delete, check properties) but doesn't read or write file content. To read or write content, you need to use classes like FileInputStream, FileOutputStream, FileReader, FileWriter

Constructors:

1. File(String pathname): Creates a File instance for the specified pathname. This can be an absolute or relative path.

Ex:

```
File file = new File("example.txt"); // Relative path
File file2 = new File("C:/user/docs/example.txt"); // Absolute path
```

2. File(String parent, String child): Creates a File instance by combining a parent directory with a child file or directory.

```
File file = new File("/user/docs", "example.txt"); // Combines parent and child path
```

This is equivalent to calling:

```
File parent = new File("/user/docs");
 File file = new File(parent, "example.txt");
```

3. File(File parent, String child): Creates a File instance from a parent File object and a child pathname string. Useful when you already have a File object representing a directory.

```
File parent = new File("/user/docs");
File file = new File(parent, "example.txt"); // Combines parent File object with child path
```

Ex:

```
import java.io.*;
class FileConstructorsExample  {
   public static void main(String[] args) {
      // 1. File(String pathname)
      File file1 = new File("File1.txt");
      File file2 = new File("F:/JAVA2024-25/Unit4/IO/Files/File2.txt");

      // 2. File(String parent, String child)
      File file3 = new File("F:/JAVA2024-25/Unit4/IO/Files","File3.txt");
      // 3. File(File parent, String child)
```

```
        File parentDir = new File("F:/JAVA2024-25/Unit4/IO/Files");
        File file4 = new File(parentDir, "File4.txt");

        // Display file paths
        System.out.println("file1 path: " + file1.getPath());
        System.out.println("file2 path: " + file2.getPath());
        System.out.println("file3 path: " + file3.getPath());
        System.out.println("file4 path: " + file4.getPath());
    }
}
```

Output:

```
file1 path: File1.txt
file2 path: F:\JAVA2024-25\Unit4\IO\Files\File2.txt
file3 path: F:\JAVA2024-25\Unit4\IO\Files\File3.txt
file4 path: F:\JAVA2024-25\Unit4\IO\Files\File4.txt
```

Methods:

Purpose	Method and Description
File and Directory creation	**createNewFile():** atomically creates a new, empty file named by this abstract pathname if and only if a file with this name does not yet exist. **mkdir()** - Creates a directory **mkdirs()** – Creates a nested directories

YADLAPATI

Ex: Creation of file, directory and nested directories

```
import java.io.*;
 class Creation_FILE_DIRECTORY {
   public static void main(String[] args) {

      try {
        File file = new File("example.txt");
        if (file.createNewFile()) {
            System.out.println("File created: " + file.getName());
        }
         else {
            System.out.println("File already exists.");
        }
      } catch (IOException e) {
        e.printStackTrace();
      }
      // Creating a directory
      File dir = new File("PVP");
```

```
        if (dir.mkdir()) {
           System.out.println("Directory created  " );
        } else {
           System.out.println("Directory already exists.");
        }

                    // Creating a nested directories
        File dir1 = new File("SAGTE/PVP/IT/II");
                    if (dir1.mkdirs()) {
           System.out.println("Nested Directories Created");
        } else {
           System.out.println("Not Created");
        }
      }
   }
```

Purpose	Method and Description
File properties	boolean exists() String getName() String getPath() String getAbsolutePath() int length() Date lastModified() boolean canRead() boolean canWrite() boolean canExecute() boolean isFile() boolean isDirectory()

SURESH YADLAPATI

Example Program:

```
import java.io.*;
 class File_Methods_Demo {
   public static void main(String[] args) {
                File file = new File("FileConstructorsExample.java");
                System.out.println("File name: " + file.getName());
                System.out.println("Absolute path: " + file.getAbsolutePath());
                System.out.println("File size (bytes): " + file.length());
                System.out.println("Is file writable? " + file.canWrite());
                System.out.println("Is file readable? " + file.canRead());
                System.out.println("Is Executable File? " + file.canExecute());
                System.out.println("Is it a directory? " + file.isDirectory());
                System.out.println("Is it a file? " + file.isFile());
                System.out.println("Last modified: " + new
java.util.Date(file.lastModified()));
```

```
        }
    }
```

Output:

```
File name: FileConstructorsExample.java
Absolute path: F:\JAVA2024-
25\Unit4\IO\Files\FileConstructorsExample.java
File size (bytes): 849
Is file writable? true
Is file readable? true
Is Executable File? true
Is it a directory? false
Is it a file? true
Last modified: Tue Nov 05 06:11:52 IST 2024
```

Example program on listing Directory:

```
import java.io.*;
 class Listing {
    public static void main(String[] args) {
                 File directory = new File("F:/JAVA2024-25/Unit4/IO/Files");
                String[] files = directory.list();
                if (files != null) {
                        for (String filename : files) {
                                System.out.println("File in directory: " + filename);
                        }
                }
        }
 }
```

Output:

```
File in directory: example.txt

File in directory: FileConstructorsExample.class

File in directory: FileConstructorsExample.java

File in directory: File_Methods_Demo.class

File in directory: File_Methods_Demo.java

File in directory: Listing.class

File in directory: Listing.java
```

FileInputStream class:

- The FileInputStream class in Java, part of the java.io package, is used to read data from a file in the form of a stream of bytes.
- It's suitable for reading binary files but can also handle text files.

Constructors

1. FileInputStream(File file): Creates a FileInputStream by opening a connection to the specified file.
2. FileInputStream(String pathname): Creates a FileInputStream by opening a connection to the file specified by the path.

Key Methods in FileInputStream

- int read(): Reads a single byte from the file. Returns the byte as an integer or -1 if the end of the file is reached.
- int read(byte[] b): Reads bytes from the file and stores them in the specified byte array.
- int read(byte[] b, int off, int len): Reads up to len bytes into the specified array b, starting from the off position in the array.
- int available(): Returns an estimate of the number of remaining bytes that can be read.
- void close(): Closes the file input stream and releases system resources.

SURESH YADLAPATI

Example: Reading a File with FileInputStream class

```
import java.io.*;

public class FISExample {
   public static void main(String[] args) {
      FileInputStream fis = null;
      try {
         // Initialize FileInputStream for a file named "example.txt"
         fis = new FileInputStream("Listing.java");

         // Read the file byte by byte
         int content;
         while ((content = fis.read()) != -1) {
            // Print the content as characters
            System.out.print((char) content);
         }
      }
               catch (IOException e) {
         e.printStackTrace();
      }
   } }
```

O/P:

```
import java.io.*;
 class Listing {
    public static void main(String[] args)
{
      File directory = new
File("F:/JAVA2024-
25/Unit4/IO/Files");
```

FileOutputStream class:

The FileOutputStream class in Java, part of the java.io package, is used to write data to a file in the form of bytes. It's commonly used for writing binary data like images and audio files, though it can also handle plain text.

Constructors

1. **FileOutputStream(File file)**: Creates a FileOutputStream that writes to the specified File.
2. **FileOutputStream(String path)**: Creates a FileOutputStream that writes to the file specified by the path.
3. **FileOutputStream(File file, boolean append)**: Creates a FileOutputStream for the specified file, with an option to append data instead of overwriting.
4. **FileOutputStream(String path, boolean append)**: Creates a FileOutputStream for the specified path, with an option to append.

SURESH YADLAPATI

Key Methods in FileOutputStream

- **void write(int b)**: Writes a single byte to the file.
- **void write(byte[] b)**: Writes an array of bytes to the file.
- **void write(byte[] b, int off, int len)**: Writes a portion of the byte array (from off to off + len) to the file.
- **void close()**: Closes the file output stream and releases system resources.

Example program on FileOutputSream:

```
import java.io.*;
 class FOSExample {
   public static void main(String[] args) {
      String data = "Hello, PVP Siddhartha Inst. of Tech.";

      try (FileOutputStream fos = new FileOutputStream("output.txt")) {
         // Convert string to bytes and write to the file
         fos.write(data.getBytes());
         System.out.println("Data written to file.");
```

```
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

More Example Programs:

1. Copying one file content to another file. To use this program, specify the name of the source file and the destination file.

```
import java.io.*;

class CopyFile
{
    public static void main(String args[]) throws Exception
    {
        int i;
        FileInputStream fin ;
        FileOutputStream fout;

        InputStreamReader isr=new InputStreamReader(System.in);
        BufferedReader br=new BufferedReader(isr);

        System.out.println("Enter source and destination files:");
        String sname=br.readLine();
        String dname=br.readLine();

        // Attempt to open the files.
        fin = new FileInputStream(sname);
        fout = new FileOutputStream(dname);
        do {
            if((i = fin.read())!= -1)
                fout.write(i);
        } while(i != -1);
        fin.close();
        fout.close();
    }
}
```

SURESH YADLAPATI

2. Merging of two file contents into a single file

```
import java.io.*;
class Merging
{
    public static void main(String args[]) throws Exception
    {
```

```
                int i;
                FileInputStream fin1,fin2 ;
                FileOutputStream fout;

                InputStreamReader isr=new InputStreamReader(System.in);
                BufferedReader br=new BufferedReader(isr);

                System.out.println("Enter source(two file) and destination files:");
                String s1name=br.readLine();
                String s2name=br.readLine();
                String dname=br.readLine();

                // Attempt to open the files.
                fin1 = new FileInputStream(s1name);
                fin2 = new FileInputStream(s2name);
                fout = new FileOutputStream(dname);
                do {
                        if((i = fin1.read())!= -1)
                                fout.write(i);
                } while(i != -1);
                fin1.close();
                do {
                        if((i = fin2.read())!= -1)
                                fout.write(i);
                } while(i != -1);
                fin2.close();
                fout.close();
        }
}
```

SURESH YADLAPATI